CODE FILE

```
-- EMPLOYEE PAYROLL MANAGEMENT SYSTEM
-- BCA DBMS Mini Project | MySQL 8.0+
-- Run this script from top to bottom.

CREATE DATABASE IF NOT EXISTS employee_payroll_db;
USE employee_payroll_db;

-- 1. Department master table
CREATE TABLE IF NOT EXISTS Departments (
    department_id INT PRIMARY KEY AUTO_INCREMENT,
    department_name VARCHAR(60) NOT NULL UNIQUE
);

-- 2. Employee records
CREATE TABLE IF NOT EXISTS Employees (
    employee_id INT PRIMARY KEY AUTO_INCREMENT,
    employee_name VARCHAR(100) NOT NULL,
    email VARCHAR(120) NOT NULL UNIQUE,
    phone VARCHAR(15),
    hire_date DATE NOT NULL,
    department_id INT NOT NULL,
    job_title VARCHAR(60) NOT NULL,
```

```
    INDEX idx_employee_department (department_id),
    CONSTRAINT fk_employee_department
        FOREIGN KEY (department_id) REFERENCES Departments(department_id)
);

-- 3. Salary structure for each employee
CREATE TABLE IF NOT EXISTS SalaryDetails (
    salary_id INT PRIMARY KEY AUTO_INCREMENT,
    employee_id INT NOT NULL UNIQUE,
    basic_salary DECIMAL(10,2) NOT NULL CHECK (basic_salary >= 0),
    hra DECIMAL(10,2) NOT NULL DEFAULT 0 CHECK (hra >= 0),
    allowance DECIMAL(10,2) NOT NULL DEFAULT 0 CHECK (allowance >= 0),
    deduction DECIMAL(10,2) NOT NULL DEFAULT 0 CHECK (deduction >= 0),
    CONSTRAINT fk_salary_employee
        FOREIGN KEY (employee_id) REFERENCES Employees(employee_id)
        ON DELETE CASCADE
);

-- 4. Monthly payroll records
CREATE TABLE IF NOT EXISTS Payroll (
    payroll_id INT PRIMARY KEY AUTO_INCREMENT,
    employee_id INT NOT NULL,
    pay_month DATE NOT NULL COMMENT 'Use the first day of the month, e.g. 2026-10-01',
    payment_date DATE,
```

```
    payment_status ENUM('Pending', 'Paid') NOT NULL DEFAULT 'Pending',
    INDEX idx_payroll_month_status (pay_month, payment_status),
    CONSTRAINT fk_payroll_employee
        FOREIGN KEY (employee_id) REFERENCES Employees(employee_id),
    CONSTRAINT uq_employee_month UNIQUE (employee_id, pay_month)
);

-- Helpful indexes are declared inside the Employees and Payroll table definitions.

-- Sample departments
INSERT INTO Departments (department_name) VALUES
('Human Resources'),
('Information Technology'),
('Finance'),
('Marketing')
ON DUPLICATE KEY UPDATE department_name = VALUES(department_name);

-- Sample employees
INSERT INTO Employees
(employee_name, email, phone, hire_date, department_id, job_title)
VALUES
('Aarav Sharma', 'aarav@example.com', '9876500001', '2024-01-15', 2, 'Software Developer'),
('Neha Verma', 'neha@example.com', '9876500002', '2023-06-10', 1, 'HR
```

```
Executive'),

('Rohan Singh', 'rohan@example.com', '9876500003', '2022-11-01', 3,
'Accountant'),

('Simran Kaur', 'simran@example.com', '9876500004', '2025-02-20', 4,
'Marketing Executive'),

('Kabir Mehta', 'kabir@example.com', '9876500005', '2024-08-05', 2, 'System
Analyst')

ON DUPLICATE KEY UPDATE employee_name = VALUES(employee_name);


-- Sample salary details (employee IDs correspond to the sample rows in a
fresh database)

INSERT INTO SalaryDetails

(employee_id, basic_salary, hra, allowance, deduction)

SELECT employee_id,

    CASE email

      WHEN 'aarav@example.com' THEN 40000.00

      WHEN 'neha@example.com' THEN 32000.00

      WHEN 'rohan@example.com' THEN 36000.00

      WHEN 'simran@example.com' THEN 30000.00

      WHEN 'kabir@example.com' THEN 45000.00

    END,

    CASE email

      WHEN 'aarav@example.com' THEN 8000.00

      WHEN 'neha@example.com' THEN 6400.00

      WHEN 'rohan@example.com' THEN 7200.00

      WHEN 'simran@example.com' THEN 6000.00
```

```
        WHEN 'kabir@example.com' THEN 9000.00
    END,
    CASE email
        WHEN 'aarav@example.com' THEN 2500.00
        WHEN 'neha@example.com' THEN 1500.00
        WHEN 'rohan@example.com' THEN 2000.00
        WHEN 'simran@example.com' THEN 1800.00
        WHEN 'kabir@example.com' THEN 3000.00
    END,
    CASE email
        WHEN 'aarav@example.com' THEN 3000.00
        WHEN 'neha@example.com' THEN 1800.00
        WHEN 'rohan@example.com' THEN 2200.00
        WHEN 'simran@example.com' THEN 1500.00
        WHEN 'kabir@example.com' THEN 4000.00
    END
FROM Employees
WHERE email IN
('aarav@example.com','neha@example.com','rohan@example.com',
            'simran@example.com','kabir@example.com')
ON DUPLICATE KEY UPDATE
   basic_salary = VALUES(basic_salary),
   hra = VALUES(hra),
   allowance = VALUES(allowance),
   deduction = VALUES(deduction);
```

```
-- Sample payroll months
INSERT INTO Payroll (employee_id, pay_month, payment_date,
payment_status)
SELECT employee_id, '2026-10-01', NULL, 'Pending'
FROM Employees
WHERE email IN
('aarav@example.com','neha@example.com','rohan@example.com',
        'simran@example.com','kabir@example.com')
ON DUPLICATE KEY UPDATE pay_month = VALUES(pay_month);


-- VIEW 1: Monthly salary slip with calculated gross and net salary
CREATE OR REPLACE VIEW SalarySlip AS
SELECT
  e.employee_id,
  e.employee_name,
  d.department_name,
  e.job_title,
  p.pay_month,
  s.basic_salary,
  s.hra,
  s.allowance,
  (s.basic_salary + s.hra + s.allowance) AS gross_salary,
  s.deduction,
  (s.basic_salary + s.hra + s.allowance - s.deduction) AS net_salary,
```

```
    p.payment_status,
    p.payment_date
FROM Employees e
JOIN Departments d ON e.department_id = d.department_id
JOIN SalaryDetails s ON e.employee_id = s.employee_id
JOIN Payroll p ON e.employee_id = p.employee_id;

-- VIEW 2: Department-wise salary summary
CREATE OR REPLACE VIEW DepartmentSalarySummary AS
SELECT d.department_name,
       COUNT(e.employee_id) AS employee_count,
       COALESCE(SUM(s.basic_salary + s.hra + s.allowance - s.deduction), 0) AS
total_net_salary
FROM Departments d
LEFT JOIN Employees e ON d.department_id = e.department_id
LEFT JOIN SalaryDetails s ON e.employee_id = s.employee_id
GROUP BY d.department_id, d.department_name;

-- DEMO QUERIES
-- A. View all employees with departments
SELECT e.employee_id, e.employee_name, e.job_title, d.department_name
FROM Employees e
INNER JOIN Departments d ON e.department_id = d.department_id
ORDER BY e.employee_id;
```

```
-- B. Generate salary slips

SELECT * FROM SalarySlip ORDER BY employee_id;


-- C. Employees earning net salary above 40000

SELECT employee_name, net_salary

FROM SalarySlip

WHERE net_salary > 40000

ORDER BY net_salary DESC;


-- D. Department-wise average net salary (aggregate + GROUP BY)

SELECT d.department_name,

      ROUND(AVG(s.basic_salary + s.hra + s.allowance - s.deduction), 2) AS
average_net_salary

FROM Departments d

JOIN Employees e ON e.department_id = d.department_id

JOIN SalaryDetails s ON s.employee_id = e.employee_id

GROUP BY d.department_name

HAVING AVG(s.basic_salary + s.hra + s.allowance - s.deduction) > 30000

ORDER BY average_net_salary DESC;


-- E. LEFT JOIN: include departments even if they have no employees

SELECT d.department_name, e.employee_name

FROM Departments d

LEFT JOIN Employees e ON e.department_id = d.department_id

ORDER BY d.department_name;
```

```
-- F. Mark one employee's October payroll as paid (transaction example)

START TRANSACTION;

UPDATE Payroll

SET payment_status = 'Paid', payment_date = '2026-10-29'

WHERE employee_id = (SELECT employee_id FROM Employees WHERE email =
'aarav@example.com')

  AND pay_month = '2026-10-01';

COMMIT;


-- G. Verify payment status

SELECT employee_name, pay_month, payment_status, payment_date

FROM SalarySlip

WHERE pay_month = '2026-10-01'

ORDER BY employee_id;


-- H. Rollback demonstration (this change will not be saved)

START TRANSACTION;

UPDATE SalaryDetails SET deduction = deduction + 100

WHERE employee_id = (SELECT employee_id FROM Employees WHERE email =
'neha@example.com');

ROLLBACK;


-- I. Index use / query plan

EXPLAIN SELECT * FROM Employees WHERE department_id = 2;
```

```
-- J. Security example (run only if your MySQL account has permission)

-- CREATE USER 'payroll_viewer'@'localhost' IDENTIFIED BY
'ChangeThis_StrongPassword!';

-- GRANT SELECT ON employee_payroll_db.* TO 'payroll_viewer'@'localhost';

-- SHOW GRANTS FOR 'payroll_viewer'@'localhost';

-- REVOKE SELECT ON employee_payroll_db.* FROM
'payroll_viewer'@'localhost';
```